

Pedro da Costa Abreu Junior

Compiler & Formal Verification Engineer

Brasília, Brazil · pedro@typetheoryforall.com · pedroabreu0.github.io · github.com/pedrotst · linkedin.com/in/pedroabreu0 · typetheoryforall.com

Summary

Compiler and formal verification engineer with an M.S. in Computer Science from Purdue University and a POPL 2023 publication. Over more than seven years I have worked on machine-checked correctness for real systems: verification of production C at Galois, a RISC-V floating-point unit at SiFive, OCaml-to-Rocq translation for Tezos protocol code at Nomadic Labs, smart-contract semantics, and security protocols. My recent work is a compiler to x86-64 verified in Rocq whose proofs were generated by AI agents under a specification I designed to be non-negotiable. I work in Rocq, Lean 4 and Isabelle, and implement in OCaml, Haskell, C, C++, Python and Rust. I also host *Type Theory Forall*, a podcast on proof assistants and programming languages, and mentor students entering the field.

Interests

- **Verified compilation and machine-level semantics:** proving compilers and low-level code correct against formal models of real instruction sets, and the trusted base those proofs rest on.
- **Proof engineering at scale:** proof automation, metaprogramming, and library design that make whole classes of proofs routine rather than bespoke.
- **Trustworthy machine-generated artifacts:** how to specify programs and proofs produced by AI systems so that the guarantees they come with stay meaningful.
- **Type theory and logic:** dependent types, GADTs and type-indexed representations, definability and incompleteness, and the relative strength of type theories.
- **Formal languages and program analysis:** grammars, automata, parsing, and analysis over compiler intermediate representations.

Education

M.S. in Computer Science, Purdue University

2018–2024

West Lafayette, IN, USA

Advisor: Prof. Benjamin Delaware. Thesis: *A Translation of OCaml GADTs into Coq*. Research in programming languages, type theory, and mechanized metatheory; graduate coursework in compilers (Prof. Tiark Rumpf), reasoning about programs (Prof. Roopsha Samanta), program analysis, networking, and data mining.

B.Sc. in Computer Science, University of Brasília

2012–2017

Brasília, Brazil

Honours thesis advised by Prof. Rodrigo Bonifácio: *Mechanizing Feature Featherweight Java with Coq* — specification of a core calculus for feature-oriented software product lines and a mechanized proof of its type safety.

OPLSS 2016 (Eugene, OR) · DeepSpec 2018 (Princeton) · Proofs and Computation 2024 (Fischbachau) · Proof Society Summer School 2024 (Birmingham)

Summer schools

Publication

A Type-Based Approach to Divide-and-Conquer Recursion in Coq

POPL 2023

Pedro Abreu, Benjamin Delaware, Alex Hubers, Christa Jenkins, J. Garrett Morris, and Aaron Stump.

Proceedings of the ACM on Programming Languages 7 (POPL), Article 3, 2023. doi.org/10.1145/3571196. Authors listed alphabetically.

My contribution: co-designed the type-based interface and developed the Rocq/Coq mechanization, including the Ltac and MetaCoq automation and the executable case studies.

Talks and Presentations

From Turing to Type Theory: The Rich Historical Context of Computation

Summer 2024

Invited talk at the University of Brasília, UFMG, UFPE, and the FP Meeting in Gothenburg. [slides]

A Translation of OCaml GADTs into Coq

May–Summer 2024

M.S. defense at Purdue, followed by invited talks at Chalmers University of Technology, Utrecht University, TU Delft, the Galinette team (Nantes), and the Cambium team (Inria Paris). [slides]

Poster: How Small Can We Make a Useful Type Theory?

POPL 2020

Student poster at POPL 2020. [poster]

Datatypes in Coq

2020

Internal training presentation for the coq-of-ocaml team at Nomadic Labs. [slides]

Experience

Independent Consultant — Verified Compilation

2026

Remote

- Delivered a compiler from a stack bytecode to x86-64, proved correct in Rocq against Jasmin’s formal model of the instruction set: sixteen correctness statements over roughly 380 lemmas, covering functional correctness, divergence preservation, fault freedom, memory footprint, the System V calling convention, and backward simulation.
- Directed AI coding agents to produce the proofs. The work that was mine was the design: the choice of Jasmin over

- CompCert on carry-flag semantics, a statement file that admits no proof vocabulary so a generated proof cannot weaken its own theorem, a satisfiability proof for the initial state relation to rule out vacuous theorems, and a **Print Assumptions** audit with the residual trusted base documented.
- Built 29 differential end-to-end tests comparing the Rocq evaluator against native binaries linked through a hand-written assembly ABI shim, byte for byte.
- Separately reviewed an ~11,600-line OCaml compiler organised around typed intermediate representations, identifying semantic inconsistencies between passes, abstraction leaks, and division-semantics mismatches.

Security Consultant, Free & Fair

Nov 2024 – Mar 2025

Remote

- Formal specification of an end-to-end verifiable internet voting protocol, using Tamarin and ProVerif for symbolic protocol analysis against an active adversary, and Lando and Clafer for system specification.

Proof Engineering Consultant, Pruvendo

2023

Remote

- Developed Rocq/Coq definitions for reasoning about communication among distributed smart contracts on the EverScale blockchain platform.

Graduate Student Researcher and Teaching Assistant, Purdue University

2018 – 2024

West Lafayette, IN, USA

- Research on type-directed program translation: translating OCaml programs with generalized algebraic datatypes into Rocq, preserving typing information; mechanized the source calculus, target calculus, and a three-phase translation using Autosubst 2.
- Built the Rocq mechanization and proof automation for the POPL 2023 recursion interface.
- Six teaching assignments across programming languages (CS 456, CS 565) and introductory programming in Java (CS 180) and C (CS 240). **2019 ACM Graduate Teaching Assistant Award.**

R&D Consultant, Nomadic Labs

May – Dec 2020

Paris, France (remote)

- Worked with the `coq-of-ocaml` team on translating OCaml into Rocq for verification of Tezos protocol code, implementing type-directed translation passes over OCaml's typed representation with Merlin.
- 8 merged pull requests** upstream, adding support for existential types and tagged GADTs. The work continued as my M.S. thesis.

Research Intern, Galois, Inc.

May – Aug 2019

Portland, OR, USA

- Specified and proved safety properties of Amazon's s2n TLS implementation and RTI's C programs in SAWScript, by symbolic execution over LLVM bitcode, including a merged fix to s2n's Cryptol specification of the TLS handshake state machine.
- Turned informal API contracts into precise pre- and postconditions over pointer and buffer layouts, and diagnosed failing verification conditions in production code owned by another organisation.
- Wrote Coq specifications for ElectionGuard, Microsoft's end-to-end verifiable voting toolkit.

Proof Engineer Intern, SiFive — Kami Team

Apr – Jul 2018

San Mateo, CA, USA

- Formalized a RISC-V floating-point unit in Rocq/Coq with the Kami hardware-verification framework and worked on its correctness properties; read the existing Chisel RTL and checked properties against it with Cadence JasperGold.

Earlier Research and Engineering

2014 – 2017

Brasília, Brazil / NICTA, Australia

- Tribunal de Contas da União** (2015–2017): development and maintenance of automated software-testing infrastructure.
- NICTA** (2014–2015): contributed to file-system verification in Isabelle.
- University of Brasília** (2014): extended Coq's rewriting tactics as an undergraduate researcher.

Selected Projects

TarskiTruth — Mathematical Logic in Lean 4

2026 – present

github.com/pedrotst/TarskiTruth — in progress

First-order arithmetic built from the syntax upward — well-formedness, substitution, evaluation, definability, Gödel coding, and diagonalization — toward Tarski's undefinability theorem, deliberately independent of Mathlib's purely semantic `ModelTheory`.

Coquedille — Rocq-to-Cedille Translator

2019 – 2020, 2026

github.com/pedrotst/coquedille

Translator between two type theories built on the MetaRocq reflection API, with a test suite that extracts the translator to Haskell and diffs rendered output on CI. Written in 2020 and ported in 2026 onto Rocq 9.2 and MetaRocq 1.5.1. Related ongoing work mechanizes CDLE, Cedille's type theory, in Rocq to compare the proof-theoretic strength of the two systems.

Program Analysis and Compilers

2019 – 2023

Purdue University — github.com/pedrotst/cs510

An LLVM analysis pass in C++ that enumerates control-flow paths to detect leaks; a Valgrind tool in C that instruments VEX IR to trace data dependencies through running binaries; a compiler, virtual machine, and runtime for MiniScala with x86-64 code generation; verification and synthesis exercises in Z3/SMT-LIB, Dafny, Sketch, and Microsoft PROSE.

Java Virtual Machine in C++11

2016 – 2017

github.com/pedrotst/JVM

Led a five-person team building a JVM: class-file reader, constant-pool parsing, bytecode verifier, heap and stack frames, and an opcode-dispatch interpreter.

Systems, emulation, and data tooling

Other

- **NES emulator in Rust** — 6502 instruction semantics, addressing modes, bus and memory mapping, instruction-level tracing (github.com/pedrotst/nes-emulator).
- **Networked systems in C** — a real-time audio streaming daemon and player, socket client and server with access control, and a Unix shell (github.com/pedrotst/cs536).
- **Machine learning in Python** — a from-scratch Naive Bayes classifier and PyTorch experiments including a GAN (github.com/pedrotst/cs573).
- **MewtwoProject** (2015, with Rodrigo Werberich) — web scraping and a SQLite data layer for a Pokémon team-building tool in Python (github.com/pedrotst/MewtwoProject).

Open Source

- [formal-land/rocq-of-ocaml](https://github.com/formal-land/rocq-of-ocaml) — 8 merged pull requests (existential types, tagged GADTs).
- [leanprover/cslib](https://github.com/leanprover/cslib) — pull request proposing InsertionSort with running-time analysis (#343).
- *Design Patterns for Humans* — Brazilian Portuguese translation; 7 merged pull requests, fourth most prolific contributor.

Community, Teaching, and Service

Founder and Host, *Type Theory Forall*

2020 – present

Long-form interview podcast on type theory, proof assistants, compilers, and formal methods, with guests from across the programming-languages community; 500+ Discord members and 4,000+ followers. Self-hosted end to end, including a hand-built RSS feed so the episodes never depend on a third-party platform, with a merchandise store run under permission from the communities whose brands it uses.

Mentoring

2022 – present

ACM SIGPLAN-M mentor (2022–2024); one-to-one tutoring through *Software Foundations*; founder of a mentoring programme helping students worldwide apply to graduate programmes in functional programming, interactive theorem proving, and type theory.

Teaching

2013 – 2024

Purdue University, six assignments (2018–2024), 2019 ACM Graduate Teaching Assistant Award. University of Brasília (2013–2016): teaching assistant for introductory computer science; rebuilt a failing course with one collaborator and coordinated a team of 20 teaching assistants.

Artifact Evaluation Committees

2022, 2025

PLDI '22 and ICFP '25 — evaluating reproducibility, soundness, and documentation of research artifacts.

Research visits and conferences

2024 – 2025

Research visits with talks at Chalmers University of Technology, Utrecht University, TU Delft, the Galinette team (Nantes), and Inria Paris; visit to UFMG. Attended ICFP (Milan) and FSE (Porto de Galinhas).

Skills

Proof assistants: Rocq/Coq (Ltac, MetaCoq/MetaRocq, extraction, `mathcomp`, Autosubst 2), Lean 4, Isabelle.

Verification: SAWScript, Cryptol, Jasmin, Kami, Cadence JasperGold, Z3/SMT-LIB, Dafny, Tamarin, ProVerif, Alloy, symbolic execution.

Programming: OCaml, Haskell, C, C++, Python, Rust, Scala, Java, Standard ML.

Language implementation: parsers and grammars, typed ASTs and IRs, type-directed translation, operational and static semantics, x86-64 code generation, bytecode interpreters, LLVM and VEX IR analysis.

Languages: Portuguese (native), English (fluent), Spanish (intermediate), French (basic).